

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования
**«Белорусский государственный университет
информатики и радиоэлектроники»**

Факультет компьютерных систем и сетей
Кафедра программного обеспечения информационных технологий

ОТЧЁТ-ИНСТРУКЦИЯ

по лабораторным работам №1 и №2
по дисциплине «Стили и методы программирования»

**Тема: «Инструкция по установке, настройке и эксплуатации
RStudio. Подготовка и выполнение скриптов на языке R»**

Выполнил:

Лебедевич Артем Владимирович
магистрант факультета
компьютерных систем и сетей
группы 556301

Преподаватель:

Парамонов Антон Иванович
кандидат технических наук, доцент
заведующий кафедрой информацион-
ных
систем и технологий ИТТ БГУИР

Минск, 2026

Содержание

Введение	3
1 ИНСТРУКЦИЯ ПО УСТАНОВКЕ, НАСТРОЙКЕ И ЭКСПЛУАТАЦИИ RSTUDIO	4
1.1 Установка R	4
1.2 Установка RStudio	4
1.3 Создание проекта	4
1.4 Подготовка и запуск R-скрипта	5
2 ИНТЕРФЕЙС RSTUDIO: ЧТО ГДЕ НАХОДИТСЯ И ЗАЧЕМ НУЖНО	6
3 ЛАБОРАТОРНАЯ РАБОТА №1: ПОШАГОВОЕ ВЫПОЛНЕНИЕ	9
3.1 Шаг 1. Создание числового вектора	9
3.2 Шаг 2. Назначение имён элементам вектора	9
3.3 Шаг 3. Построение столбиковой диаграммы	9
3.4 Шаг 4. Создание строкового вектора	10
3.5 Шаг 5. Сортировка оценок и согласование имён	11
3.6 Шаг 6. Добавление легенды к графику	11
3.7 Шаг 7. Создание матрицы	12
3.8 Шаг 8. Построение мозаичной диаграммы	12
3.9 Шаг 9. Создание фрейма данных и итоговая статистика	13
4 МЕТОД УКАЗАНИЯ ПОДГОТОВКИ СКРИПТА ДЛЯ ВЫПОЛНЕНИЯ ЛР 2	14
4.1 Какие библиотеки нужны	14
4.2 Порядок подготовки файлов	15
5 ЛАБОРАТОРНАЯ РАБОТА №2: МЕТОДЫ, ПАРАМЕТРЫ И ВЛИЯНИЕ НА РАБОТУ	16
5.1 Скрипт 1. Подключение к API и сохранение данных	16
5.2 Скрипт 2. Загрузка, анализ и визуализация	18
6 ПРАКТИЧЕСКИЕ ПОДСКАЗКИ И ТИПОВЫЕ ОШИБКИ	23
Заключение	24
Список использованных источников	25

ВВЕДЕНИЕ

Данный документ подготовлен как практическая инструкция к лабораторным работам №1 и №2 по языку R. Его задача не просто повторить листинг программы, а объяснить, что именно делается на каждом шаге, зачем используется тот или иной метод, какие параметры ему передаются и как эти параметры влияют на результат.

Лабораторная работа №1 посвящена знакомству с R и средой RStudio: созданию проекта, запуску скриптов, работе с векторами, матрицами, фреймами данных и базовыми графиками. Лабораторная работа №2 продолжает эту тему на более прикладном примере: данные загружаются из удалённого API, сохраняются в CSV-файл, затем анализируются и визуализируются с помощью внешних библиотек.

Инструкция написана с расчётом на студента, который только открывает RStudio и хочет понимать не только «что нажать», но и «почему именно так». Поэтому рядом с командами приводятся пояснения по вкладкам интерфейса, библиотекам, параметрам функций и типовым ошибкам.

1 ИНСТРУКЦИЯ ПО УСТАНОВКЕ, НАСТРОЙКЕ И ЭКСПЛУАТАЦИИ RSTUDIO

Для выполнения лабораторных работ используется связка из двух программных компонентов: языка R и среды разработки RStudio. Важно понимать разницу между ними. R является самым языком и интерпретатором, который выполняет команды. RStudio является удобной оболочкой вокруг R: в ней можно писать скрипты, запускать их по частям, смотреть переменные, строить графики и работать с файлами проекта.

1.1 Установка R

Сначала устанавливается язык R. Для этого необходимо открыть официальный сайт CRAN: <https://cran.r-project.org/>. На сайте выбирается версия для своей операционной системы: Windows, macOS или Linux.

Рекомендуемый порядок установки:

1. Скачать установщик R с CRAN.
2. Запустить установщик и оставить стандартные параметры, если нет специальных требований.
3. После установки проверить, что R запускается: открыть терминал или консоль R и выполнить команду `R.version.string`.

Главный смысл этого шага: RStudio не заменяет R. Если установить только RStudio, но не установить сам R, среда разработки не сможет выполнять скрипты.

1.2 Установка RStudio

После установки R загружается RStudio Desktop с сайта Posit: <https://posit.co/downloads/>. Для лабораторных работ достаточно бесплатной версии RStudio Desktop.

На macOS приложение обычно перетаскивается в папку **Applications**. На Windows используется обычный мастер установки. После первого запуска RStudio автоматически находит установленный интерпретатор R. Если на компьютере установлено несколько версий R, нужную версию можно выбрать в настройках RStudio.

1.3 Создание проекта

Проект в RStudio нужен для того, чтобы все скрипты, данные и результаты лежали в одной рабочей папке. Это снижает количество ошибок с путями к файлам.

Порядок создания проекта:

1. Открыть меню **File -> New Project....**

2. Выбрать **New Directory** -> **New Project**.
3. Указать имя папки проекта и место её расположения.
4. Нажать **Create Project**.

После этого RStudio создаёт файл с расширением **.Rproj**. Его удобно открывать в следующий раз двойным щелчком: RStudio сразу восстановит рабочую папку проекта.

1.4 Подготовка и запуск R-скрипта

Скрипт на языке R сохраняется в файле с расширением **.R**. Для лабораторных работ используются файлы **lab1_script.R**, **lab2_script1.R** и **lab2_script2.R**.

Основные способы запуска:

1. **Ctrl+Enter** или **Cmd+Enter** на macOS: выполнить текущую строку или выделенный фрагмент. Этот способ удобен, когда надо пройти лабораторную пошагово.
2. Кнопка **Source**: выполнить весь скрипт целиком. Этот способ удобен после проверки отдельных строк.
3. Команда **source("lab1_script.R")** в консоли: запустить файл как программу.
4. Команда **Rscript lab1_script.R** из терминала: выполнить скрипт без открытия RStudio.

Для учебной работы лучше начинать с первого варианта. Тогда видно, какая команда что создаёт, какой график появляется и на каком шаге возникла ошибка.

2 ИНТЕРФЕЙС RSTUDIO: ЧТО ГДЕ НАХОДИТСЯ И ЗАЧЕМ НУЖНО

Окно RStudio обычно разделено на четыре рабочие области. На первых занятиях кажется, что вкладок слишком много, но каждая из них решает конкретную задачу.

Таблица 1: Основные области и вкладки RStudio

Вкладка или область	Что показывает	Что можно сделать при выполнении ЛР
Source	Текст открытого R-скрипта.	Написать код, выделить нужный фрагмент, выполнить строку через Ctrl+Enter , сохранить файл.
Console	Команды R и результаты их выполнения.	Смотреть вывод <code>print()</code> , <code>cat()</code> , <code>summary()</code> , сообщения об ошибках и предупреждения.
Environment	Созданные переменные, векторы, таблицы, модели.	Проверить, появились ли <code>data1</code> , <code>A</code> , <code>df</code> , <code>users_data</code> , <code>kmeans_result</code> .
History	История ранее выполненных команд.	Быстро повторить команду, если она уже запускалась вручную.
Files	Файлы и папки текущего проекта.	Проверить наличие скриптов, CSV-файла и сохранённых PNG-графиков.
Plots	Графики, построенные командами R.	Посмотреть результат <code>barplot()</code> , <code>mosaicplot()</code> , <code>ggplot()</code> , экспортировать график.
Packages	Установленные и подключённые пакеты.	Проверить наличие <code>httr</code> , <code>jsonlite</code> , <code>ggplot2</code> , <code>dplyr</code> .
Help	Справка по функциям и пакетам.	Открыть документацию, например <code>?barplot</code> , <code>?kmeans</code> , <code>?ggsave</code> .
Viewer	HTML-результаты, интерактивные отчёты и web-содержимое.	В этих лабораторных используется редко, но полезен при работе с HTML-виджетами.

При выполнении ЛР 1 особенно важны вкладки **Source**, **Console**, **Environment** и **Plots**. Например, после команды создания вектора `data1` он появляется в **Environment**; после `barplot()` график появляется в **Plots**; после `summary(df)` результат выводится в **Console**.

Удобная логика работы такая: пишем или открываем код в **Source**, запускаем его по частям, смотрим текстовый результат в **Console**, проверяем переменные в **Environment**, графики смотрим в **Plots**. Если файл не найден, первым делом проверяется вкладка **Files** и текущая рабочая папка.

3 ЛАБОРАТОРНАЯ РАБОТА №1: ПОШАГОВОЕ ВЫПОЛНЕНИЕ

В ЛР 1 используется базовый R без подключения внешних библиотек. Это важно: все методы из данного раздела входят в стандартную поставку языка R, поэтому отдельные команды `install.packages()` и `library()` для них не нужны.

3.1 Шаг 1. Создание числового вектора

Сначала создаётся вектор оценок:

```
1 data1 <- c(8, 5, 9, 7, 3, 10, 6, 4)
```

Метод `c()` используется для объединения значений в один вектор. В R вектор является одной из базовых структур данных. Здесь вектор хранит оценки студентов.

Таблица 2: Метод `c()` в ЛР 1

Параметр	Назначение и влияние
Набор значений	Значения перечисляются через запятую. Порядок важен: первый элемент вектора считается первым студентом, второй элемент – вторым и так далее.

3.2 Шаг 2. Назначение имён элементам вектора

Далее элементам вектора присваиваются фамилии:

```
1 names(data1) <- c("Иванов", "Петров", "Сидоров", "Козлов",  
2 "Новиков", "Фёдоров", "Морозов", "Волков")
```

Метод `names()` используется для задания имён элементам объекта. После этого значения `data1` можно воспринимать не как безымянные числа, а как оценки конкретных студентов.

Таблица 3: Метод `names()`

Что указывается	Как влияет на работу
<code>names(data1)</code>	Указывает, какому объекту назначаются имена.
Вектор строк	Количество имён должно соответствовать количеству элементов <code>data1</code> . Если имён меньше или больше, данные становятся неудобными для чтения и могут появиться NA.

3.3 Шаг 3. Построение столбиковой диаграммы

Для визуализации оценок используется встроенный метод `barplot()`:

```

1 barplot(data1,
2         main = "Успеваемость студентов",
3         col = rainbow(length(data1)),
4         ylab = "Оценка",
5         las = 2,
6         cex.names = 0.8)

```

Метод `barplot()` строит столбиковую диаграмму. В данной лабораторной он нужен, чтобы быстро увидеть, у кого оценка выше, а у кого ниже.

Таблица 4: Основные параметры `barplot()`

Параметр	Назначение и влияние
<code>height</code> или первый аргумент	Набор чисел, по которым строятся столбцы. В примере это <code>data1</code> .
<code>main</code>	Заголовок графика. Помогает понять, что изображено.
<code>col</code>	Цвета столбцов. В примере <code>rainbow(length(data1))</code> создаёт столько цветов, сколько элементов в векторе.
<code>ylab</code>	Подпись вертикальной оси. Здесь ось показывает оценку.
<code>las = 2</code>	Поворачивает подписи оси, чтобы фамилии лучше помещались под столбцами.
<code>cex.names = 0.8</code>	Уменьшает размер подписей. Чем меньше значение, тем компактнее текст.

Дополнительно применяется метод `rainbow()`. Он относится к базовой графике R и создаёт палитру цветов. Параметр `length(data1)` означает, что цветов должно быть ровно столько, сколько столбцов.

3.4 Шаг 4. Создание строкового вектора

Во втором векторе хранятся имена студентов:

```

1 data2 <- c("Алексей", "Борис", "Виктор", "Григорий",
2           "Дмитрий", "Евгений", "Жан", "Захар")

```

Здесь снова используется метод `c()`, но теперь элементы являются строками. Для R это обычный вектор, только тип данных не числовой, а символьный.

3.5 Шаг 5. Сортировка оценок и согласование имён

Чтобы показать оценки от большей к меньшей, применяется сортировка:

```
1 sorted_data1 <- sort(data1, decreasing = TRUE)
2 sorted_data2 <- data2[order(data1, decreasing = TRUE)]
```

Метод `sort()` возвращает отсортированные значения. Метод `order()` возвращает порядок индексов, по которому надо переставить другой вектор. В данной работе это важно: если отсортировать только оценки, имена студентов могут перестать соответствовать своим оценкам. Поэтому `order()` используется для перестановки `data2` тем же образом.

Таблица 5: Методы сортировки

Метод	Зачем используется	Важные параметры
<code>sort()</code>	Получить значения в отсортированном виде.	<code>decreasing = TRUE</code> сортирует по убыванию, <code>FALSE</code> – по возрастанию.
<code>order()</code>	Получить индексы элементов в нужном порядке.	Используется с тем же параметром <code>decreasing</code> , чтобы связанные данные переставлялись одинаково.

3.6 Шаг 6. Добавление легенды к графику

После построения отсортированной диаграммы добавляется легенда:

```
1 legend("topright",
2       legend = sorted_data2,
3       fill = rainbow(length(sorted_data2)),
4       cex = 0.7,
5       title = "Студенты")
```

Метод `legend()` нужен, чтобы расшифровать цвета или обозначения на графике.

Таблица 6: Параметры `legend()`

Параметр	Назначение и влияние
"topright"	Положение легенды. Можно использовать также "topleft", "bottomright" и другие варианты.
legend	Текстовые подписи внутри легенды. В примере это имена студентов.
fill	Цветные прямоугольники рядом с подписями. Цвета должны соответствовать цветам столбцов.
cex	Размер текста легенды. Значение 0.7 делает подписи немного меньше.
title	Заголовок легенды.

3.7 Шаг 7. Создание матрицы

Матрица создаётся методом `matrix()`:

```

1 A <- matrix(c(2, 5, 8, 1,
2               3, 7, 4, 6,
3               9, 2, 5, 3,
4               1, 8, 6, 4,
5               7, 3, 2, 9),
6               nrow = 4, ncol = 5)
```

Метод `matrix()` преобразует набор значений в таблицу фиксированного размера. В лабораторной создаётся матрица размером 4 на 5.

Таблица 7: Параметры `matrix()`

Параметр	Назначение и влияние
Первый аргумент	Значения, из которых заполняется матрица.
nrow = 4	Количество строк. В примере это 4 группы.
ncol = 5	Количество столбцов. В примере это 5 предметов.
byrow	Если указать <code>byrow = TRUE</code> , матрица будет заполняться по строкам. По умолчанию R заполняет матрицу по столбцам.

Названия строк и столбцов задаются методами `rownames()` и `colnames()`. Это делает матрицу понятнее при выводе и при построении графика.

3.8 Шаг 8. Построение мозаичной диаграммы

Для матрицы строится мозаичная диаграмма:

```

1 mosaicplot(A,
2             main = "Мозаичная диаграмма матрицы A",
```

```

3         color = heat.colors(ncol(A)),
4         las = 1)

```

Метод `mosaicplot()` показывает соотношения между категориями в виде прямоугольников. Чем больше значение в ячейке матрицы, тем заметнее соответствующая часть диаграммы.

Таблица 8: Параметры `mosaicplot()`

Параметр	Назначение и влияние
Первый аргумент	Таблица или матрица, по которой строится диаграмма.
<code>main</code>	Заголовок диаграммы.
<code>color</code>	Цветовая схема. В примере <code>heat.colors(ncol(A))</code> создаёт тёплую палитру по числу столбцов.
<code>las = 1</code>	Настраивает ориентацию подписей осей, чтобы они легче читались.

3.9 Шаг 9. Создание фрейма данных и итоговая статистика

В конце данные объединяются в таблицу:

```

1 df <- data.frame(Имя = data2, Оценка = as.numeric(data1))
2 print(df)
3 summary(df)

```

Метод `data.frame()` создаёт табличную структуру, похожую на таблицу Excel: каждый столбец имеет имя и свой тип данных. Метод `as.numeric()` нужен, чтобы гарантировать числовой тип оценок. Метод `summary()` выводит краткую статистику: минимум, максимум, среднее, медиану и квартильные значения для числовых столбцов.

Таблица 9: Итоговые методы ЛР 1

Метод	Назначение	На что обратить внимание
<code>data.frame()</code>	Создаёт таблицу из нескольких векторов.	Длина всех столбцов должна совпадать.
<code>as.numeric()</code>	Преобразует данные к числовому типу.	Нужно для корректных расчётов и статистики.
<code>print()</code>	Выводит объект в консоль.	Удобно для проверки результата.
<code>summary()</code>	Показывает краткое описание данных.	Для чисел выводит статистику, для строк – другую сводную информацию.

4 МЕТОД УКАЗАНИЯ ПОДГОТОВКИ СКРИПТА ДЛЯ ВЫПОЛНЕНИЯ ЛР 2

Лабораторная работа №2 состоит из двух логических частей. Первый скрипт подключается к удалённому ресурсу и сохраняет данные. Второй скрипт читает сохранённый CSV-файл, выполняет анализ и строит графики.

Такое разделение удобно: если данные уже скачаны, не нужно каждый раз обращаться к API. Можно отдельно отлаживать анализ и визуализацию.

4.1 Какие библиотеки нужны

В ЛР 2 используются внешние библиотеки R. Если они ещё не установлены, их надо установить один раз:

```
1 install.packages(c("httr", "jsonlite", "ggplot2", "dplyr", "corrplot", "cluster"))
```

После установки в каждом новом сеансе R нужные библиотеки подключаются командой `library()`.

Таблица 10: Библиотеки для ЛР 2

Библиотека	Для чего используется	Где применяется
httr	Выполнение HTTP-запросов к удалённым ресурсам.	<code>GET()</code> , <code>status_code()</code> , <code>content()</code> в первом скрипте.
jsonlite	Преобразование JSON-ответа в таблицу R.	<code>fromJSON()</code> в первом скрипте.
ggplot2	Построение графиков по грамматике графики.	<code>ggplot()</code> , <code>geom_point()</code> , <code>geom_bar()</code> , <code>ggsave()</code> во втором скрипте.
dplyr	Удобная обработка таблиц.	<code>select()</code> , <code>mutate()</code> во втором скрипте.
corrplot	Визуализация корреляционных матриц.	Подключена как полезная библиотека для анализа, хотя в текущем варианте скрипта напрямую почти не используется.
cluster	Методы кластерного анализа.	Подключена для задач кластеризации; сам метод <code>kmeans()</code> доступен в базовом пакете <code>stats</code> .

4.2 Порядок подготовки файлов

Перед запуском ЛР 2 необходимо проверить структуру проекта:

1. Открыть проект `r_studio_labs.Rproj` или папку проекта в RStudio.
2. Убедиться, что файл `lab2_script1.R` находится в папке `lab2_r_analyse`.
3. Убедиться, что файл `lab2_script2.R` находится в той же папке.
4. Запустить сначала `lab2_script1.R`. После его выполнения должен появиться файл `users_data.csv`.
5. Затем запустить `lab2_script2.R`. Он читает `users_data.csv`, добавляет кластеры и сохраняет графики `cluster_scatter.png` и `cluster_bar.png`.

Ключевой момент: второй скрипт зависит от результата первого. Если файл `users_data.csv` не создан или лежит не в рабочей папке, команда `read.csv()` выдаст ошибку.

5 ЛАБОРАТОРНАЯ РАБОТА №2: МЕТОДЫ, ПАРАМЕТРЫ И ВЛИЯНИЕ НА РАБОТУ

5.1 Скрипт 1. Подключение к API и сохранение данных

В начале первого скрипта подключаются библиотеки:

```
1 library(httr)
2 library(jsonlite)
```

Метод `library()` загружает установленный пакет в текущий сеанс R. Если пакет установлен, но не подключён, его функции могут быть недоступны по короткому имени.

Таблица 11: Метод `library()`

Параметр	Назначение и влияние
Имя пакета	Указывает, какой пакет подключить. Например, <code>library(httr)</code> делает доступной функцию <code>GET()</code> .

Далее указывается адрес удалённого ресурса и выполняется GET-запрос:

```
1 url <- "https://jsonplaceholder.typicode.com/users"
2 response <- GET(url)
```

Метод `GET()` из библиотеки `httr` отправляет HTTP-запрос на сервер. В лабораторной он используется для получения списка пользователей из тестового API JSONPlaceholder.

Таблица 12: Метод `GET()`

Параметр	Назначение и влияние
<code>url</code>	Адрес ресурса. Если адрес указан неверно, сервер вернёт ошибку или запрос не выполнится.
<code>timeout()</code>	Рекомендуемый дополнительный параметр для реальных проектов. Ограничивает время ожидания ответа, чтобы программа не зависала слишком долго.
<code>add_headers()</code>	Используется, если API требует заголовки, например токен авторизации. В JSONPlaceholder авторизация не нужна.

Ответ сервера проверяется по HTTP-коду:

```
1 if (status_code(response) == 200) {
2   cat("Подключение успешно. Код ответа:", status_code(response)
3     , "\n")
4 } else {
```



```

4   stop("Ошибка подключения: ", status_code(response))
5 }

```

Метод `status_code()` возвращает код ответа сервера. Код 200 означает успешный запрос. Метод `cat()` выводит понятное сообщение в консоль, а `stop()` останавливает выполнение программы при ошибке.

После проверки ответ преобразуется в текст, а затем в таблицу:

```

1 users_json <- content(response, as = "text", encoding = "UTF-8"
2   )
  users_data <- fromJSON(users_json, flatten = TRUE)

```

Таблица 13: Получение и разбор JSON

Метод	Важные параметры	Как влияет на результат
<code>content()</code>	<code>as = "text"</code>	Ответ сервера возвращается как обычная строка JSON, которую удобно передать в <code>fromJSON()</code> .
<code>content()</code>	<code>encoding = "UTF-8"</code>	Корректно обрабатывает символы в кодировке UTF-8. Это особенно важно для текстовых данных.
<code>fromJSON()</code>	<code>flatten = TRUE</code>	Вложенные поля JSON разворачиваются в плоские столбцы, например <code>address.geo.lat</code> . Без этого вложенные структуры было бы сложнее анализировать.

В конце данные сохраняются в CSV-файл:

```

1 write.csv(users_data, file = "users_data.csv",
2   row.names = FALSE, fileEncoding = "UTF-8")

```

Метод `write.csv()` сохраняет таблицу R в файл CSV. Такой файл можно открыть не только в R, но и в табличном редакторе.

Таблица 14: Параметры `write.csv()`

Параметр	Назначение и влияние
<code>users_data</code>	Таблица, которую нужно сохранить.
<code>file = "users_data.csv"</code>	Имя выходного файла. Файл создаётся в текущей рабочей папке.
<code>row.names = FALSE</code>	Не записывает отдельный столбец с номерами строк. Для учебного CSV это удобнее и чище.
<code>fileEncoding = "UTF-8"</code>	Сохраняет файл в кодировке UTF-8, чтобы текст корректно читался на разных системах.

5.2 Скрипт 2. Загрузка, анализ и визуализация

Второй скрипт начинается с чтения CSV-файла:

```
1 users_data <- read.csv("users_data.csv", fileEncoding = "UTF-8")
```

Метод `read.csv()` загружает CSV-файл в `data.frame`. Параметр `fileEncoding = "UTF-8"` помогает избежать проблем с кодировкой.

Затем выбираются координаты пользователей:

```
1 numeric_data <- users_data %>%
2   select(address.geo.lat, address.geo.lng) %>%
3   mutate(
4     address.geo.lat = as.numeric(address.geo.lat),
5     address.geo.lng = as.numeric(address.geo.lng)
6   )
```

Оператор `%>%` из экосистемы `dplyr` читается как «передать результат дальше». Он позволяет записывать обработку данных цепочкой: сначала берём таблицу, затем выбираем нужные столбцы, затем преобразуем типы.

Таблица 15: Подготовка числовых признаков

Метод	Назначение	Важные параметры
<code>select()</code>	Оставляет только нужные столбцы.	Указываются имена столбцов: <code>address.geo.lat</code> , <code>address.geo.lng</code> .
<code>mutate()</code>	Создаёт новые столбцы или изменяет существующие.	В примере координаты перезаписываются после преобразования к числу.
<code>as.numeric()</code>	Преобразует значение в числовой тип.	Без числового типа <code>kmeans()</code> не сможет корректно посчитать расстояния между точками.

Кластеризация выполняется методом k-средних:

```

1 set.seed(42)
2 kmeans_result <- kmeans(numeric_data, centers = 3, nstart = 25)
3 users_data$cluster <- as.factor(kmeans_result$cluster)

```

Метод `kmeans()` группирует объекты так, чтобы точки внутри одного кластера были ближе друг к другу, чем к точкам других кластеров. В данной работе пользователи группируются по широте и долготе.

Таблица 16: Параметры кластеризации

Метод или параметр	Назначение и влияние
<code>set.seed(42)</code>	Фиксирует генератор случайных чисел. Это нужно, чтобы при повторном запуске получались одинаковые кластеры. Число 42 можно заменить, но тогда результат может немного измениться.
<code>kmeans()</code>	Запускает алгоритм k-средних для числовых данных.
<code>centers = 3</code>	Количество кластеров. В лабораторной выбрано 3: этого достаточно, чтобы разделить 10 пользователей на несколько групп, но не дробить данные слишком мелко.
<code>nstart = 25</code>	Количество случайных начальных запусков алгоритма. Чем больше значение, тем выше шанс найти более удачное разбиение. Для небольшой выборки 25 является хорошим практическим значением.
<code>as.factor()</code>	Превращает номер кластера в категорию. Это удобно для раскраски графиков в <code>ggplot2</code> .

Для визуализации используется `ggplot2`:

```

1 p1 <- ggplot(users_data,
2               aes(x = as.numeric(address.geo.lng),
3                   y = as.numeric(address.geo.lat),
4                       color = cluster)) +
5   geom_point(size = 4) +
6   geom_text(aes(label = name), vjust = -1, size = 3) +
7   labs(title = "Кластеризация пользователей по геолокации",
8         x = "Долгота (lng)",
9         y = "Широта (lat)",
10        color = "Кластер") +
11   theme_minimal()

```

Метод `ggplot()` создаёт основу графика, а остальные методы добавляют слои и оформление.

Таблица 17: Методы `ggplot2` для диаграммы рассеяния

Метод	Для чего используется	Важные параметры
<code>ggplot()</code>	Создаёт объект графика.	Первый аргумент – таблица данных.
<code>aes()</code>	Задаёт соответствия между данными и внешним видом графика.	<code>x</code> – долгота, <code>y</code> – широта, <code>color = cluster</code> раскрашивает точки по кластерам.
<code>geom_point()</code>	Добавляет точки на график.	<code>size = 4</code> задаёт размер точек.
<code>geom_text()</code>	Добавляет подписи к точкам.	<code>label = name</code> подписывает точки именами пользователей, <code>vjust = -1</code> поднимает подпись выше точки.
<code>labs()</code>	Задаёт заголовок, подписи осей и легенды.	Чем понятнее подписи, тем легче читать результат.
<code>theme_minimal()</code>	Применяет аккуратную минималистичную тему.	Убирает лишнюю визуальную нагрузку.

Для распределения пользователей по кластерам строится столбиковая диаграмма:

```

1 p2 <- ggplot(users_data, aes(x = cluster, fill = cluster)) +
2   geom_bar() +
3   labs(title = "Распределение пользователей по кластерам",
4         x = "Кластер",
5         y = "Количество пользователей") +
6   theme_minimal()

```

Метод `geom_bar()` сам подсчитывает количество наблюдений в каждой категории `x`. Поэтому отдельно считать число пользователей по кластерам не требуется.

В конце графики сохраняются:

```

1 ggsave("cluster_scatter.png", plot = p1, width = 10, height =
   7, dpi = 150)
2 ggsave("cluster_bar.png", plot = p2, width = 8, height = 6, dpi
   = 150)

```

Метод `ggsave()` сохраняет последний или явно указанный график в файл.

Таблица 18: Параметры `ggsave()`

Параметр	Назначение и влияние
Имя файла	Расширение определяет формат: <code>.png</code> , <code>.pdf</code> , <code>.jpg</code> .
<code>plot</code>	Указывает, какой график сохранить. Без этого параметра сохраняется последний выведенный график.
<code>width, height</code>	Размер изображения. Чем больше значения, тем крупнее итоговый файл и свободнее расположение подписей.
<code>dpi = 150</code>	Разрешение изображения. Большее значение даёт более чёткую картинку, но увеличивает размер файла.

6 ПРАКТИЧЕСКИЕ ПОДСКАЗКИ И ТИПОВЫЕ ОШИБКИ

Таблица 19: Типовые проблемы при выполнении ЛР 1 и ЛР 2

Проблема	Почему возникает	Как исправить
<code>there is no package called ...</code>	Пакет не установлен.	Выполнить <code>install.packages("имя_пакета")</code> , затем снова <code>library()</code> .
<code>could not find function ...</code>	Пакет установлен, но не подключён, либо в имени функции опечатка.	Проверить строку <code>library(...)</code> и имя метода.
<code>cannot open file 'users_data.csv'</code>	Второй скрипт запущен раньше первого или рабочая папка выбрана неверно.	Сначала запустить <code>lab2_script1.R</code> , затем проверить файл во вкладке Files .
График не появился	Выполнена не вся команда или график построен, но вкладка Plots скрыта.	Перейти во вкладку Plots , выполнить команду <code>print(p1)</code> или <code>print(p2)</code> .
<code>kmeans()</code> выдаёт ошибку по типам данных	Координаты прочитались как текст, а не как числа.	Использовать <code>as.numeric()</code> для столбцов широты и долготы.
CSV создался не там, где ожидалось	Текущая рабочая папка отличается от папки со скриптом.	Проверить <code>getwd()</code> или открыть проект через файл <code>.Rproj</code> .

Перед сдачей удобно пройти короткий чек-лист:

1. В **Console** нет красных сообщений об ошибках.
2. Во вкладке **Environment** видны основные объекты: `data1`, `df`, `users_data`, `kmeans_result`.
3. Во вкладке **Files** есть `users_data.csv`, `cluster_scatter.png`, `cluster_bar.png`.
4. Во вкладке **Plots** отображаются построенные графики.
5. В отчёте указано, какие методы применялись, какие параметры были выбраны и почему.

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторных работ №1 и №2 студент получает не только набор готовых R-скриптов, но и понимание процесса работы в RStudio. В ЛР 1 осваиваются базовые структуры данных языка R и встроенные средства визуализации. В ЛР 2 к ним добавляются внешние библиотеки, загрузка данных из API, сохранение CSV-файла, кластеризация методом k-средних и построение графиков через `ggplot2`.

Главная практическая идея такая: любой скрипт лучше выполнять не вслепую, а по шагам. После каждой важной команды надо смотреть, что изменилось в **Console**, **Environment**, **Files** и **Plots**. Тогда лабораторная превращается не в переписывание кода, а в понятный процесс анализа данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 The Comprehensive R Archive Network [Электронный ресурс]. – Режим доступа: <https://cran.r-project.org/>. – Дата доступа: 15.06.2026.
- 2 Posit. RStudio Desktop [Электронный ресурс]. – Режим доступа: <https://posit.co/downloads/>. – Дата доступа: 15.06.2026.
- 3 RStudio IDE User Guide [Электронный ресурс]. – Режим доступа: <https://docs.posit.co/ide/user/>. – Дата доступа: 15.06.2026.
- 4 JSONPlaceholder – Free Fake REST API [Электронный ресурс]. – Режим доступа: <https://jsonplaceholder.typicode.com/>. – Дата доступа: 15.06.2026.
- 5 Wickham, H. ggplot2: Elegant Graphics for Data Analysis / H. Wickham. – 2nd ed. – New York : Springer, 2016. – 260 p.
- 6 Wickham, H. R for Data Science / H. Wickham, G. Grolemund. – Sebastopol : O'Reilly Media, 2017. – 492 p.